Practical Exam - Turn Based RPG

R.A. Campita
Cavite State University, 07.10.2026

1. Objectives

Games like sword-fighting RPGs are built from objects that know things and do things. In this exam, you will model a small **turn-based RPG** using everything you know about OOP so far. By the end of the exam, you should have been able to:

- Create a program that uses all concepts explored in the laboratory classes prior to the exam
  - Methods
  - Classes and Objects
  - Inheritance
- Design your classes so that similar classes share code instead of repeating it

You are graded on how your game **behaves** and on how little code you repeat. There is no scaffolding: you decide which classes, fields, and methods to write, as long as `Main` compiles and runs.

2. Instructions

Create a Java program where a player fights waves of enemies. You have two weapons (a **sword** and a **staff**), but each weapon only hurts certain enemies: the sword hurts wizards, and the staff hurts knights. On every enemy turn, the enemy asks you a math question. If you answer correctly, you take no damage. Otherwise, the enemy attacks you. You lose when your HP reaches 0. You win when every enemy in both waves is defeated.

The full `Main` class is provided below. **Do not modify it.** It drives the whole game, so your classes must work with it as written.

Main.java

```java
import java.util.Scanner;

class Main {
  public static void main(String[] args) {
    System.out.println("Turn Based RPG");
    System.out.println("--------------");
    System.out.println("[1] Play");
    System.out.println("[2] Quit");

    Scanner sc = new Scanner(System.in);
    String input = sc.nextLine();

    switch (input) {
      case "1":
        playGame(sc);
        break;
      case "2":
        break;
    }

    sc.close();
  }

  static void playGame(Scanner sc) {
    Player player = new Player("Steve");
    player.showWeapons();

    // Wave 1: a single knight
    player.seeEnemy(new Knight());
    battle(player, sc);

    // Wave 2: a knight and two wizards
    player.seeEnemy(new Knight());
    player.seeEnemy(new Wizard());
    player.seeEnemy(new Wizard());
    battle(player, sc);

    if (player.hp <= 0) {
```

```
39        System.out.println("You lose...");
40      } else {
41        System.out.println("You win!!!");
42      }
43    }
44
45    // One wave of combat. Players and enemies take turns until the
      player
46    // is dead or every visible enemy is gone.
47    static void battle(Player player, Scanner sc) {
48      while (!player.visibleEnemies.isEmpty() && player.hp > 0) {
49        player.playerTurn(sc);
50        for (Enemy enemy : player.visibleEnemies) {
51          enemy.enemyTurn(player, sc);
52        }
53      }
54    }
55  }
```

2.1. What `Main` needs from you

`Main` only fixes a few names. Everything else is up to you.

- Classes named `Player`, `Enemy`, `Knight`, and `Wizard` (`Knight` and `Wizard` objects must be usable wherever an `Enemy` is expected).
- `new Player(String name)`, with an `int hp` and an `ArrayList<Enemy> visibleEnemies` you can read directly.
- `Player` methods `showWeapons()`, `seeEnemy(Enemy)`, and `playerTurn(Scanner)`.
- An `Enemy` method `enemyTurn(Player, Scanner)` that works for every kind of enemy.
- `Knight()` and `Wizard()` constructors with no arguments.

Notice how `battle` iterates over `player.visibleEnemies`. The loop only knows it is holding `Enemy` objects, but each call to `enemy.enemyTurn(player, sc)` must still behave according to the kind of enemy it is: a knight asks a subtraction question and a wizard asks a multiplication question. Similar to the AST exercise, we will use some polymorphism in this exam as well.

3. Game Rules

Thing	Rules
Player	Named „Steve“. Starts with 3 HP, a sword, and a staff. Each weapon does 1 damage.

Thing	Rules
Knight	4 HP, 1 damage. Only the **staff** hurts it. Asks a **subtraction** question with two random numbers from 0 to 99, in the form `What is 12 - 45?`.
Wizard	2 HP, 1 damage. Only the **sword** hurts it. Asks a **multiplication** question with two random numbers from 0 to 9, in the form `What is 3 * 7?`.
Waves	Wave 1 is a single knight. Wave 2 is a knight and two wizards (already set up by `Main`).

Each round has a player turn, then every enemy still alive takes a turn:

- **Player turn:** the player picks an enemy from a numbered menu, then picks a weapon from a numbered menu. The attack is always announced. If the weapon can't hurt that enemy, a message says so and nothing else happens. Otherwise, the enemy loses HP equal to the weapon's damage.
- **Enemy defeated:** when an enemy's HP reaches 0 or less, a message says it is defeated and it no longer appears in menus or takes turns.
- **Enemy turn:** the enemy asks a **new** random question. A correct answer means no damage. A wrong answer means the enemy attacks, and the player loses HP equal to the enemy's damage. The remaining HP is shown.
- **Player defeated:** the player's HP never shows below 0. Once the player has 0 HP, no enemy asks any more questions or attacks, even if other enemies are still alive in the same wave.

You may assume the player always types valid numbers.

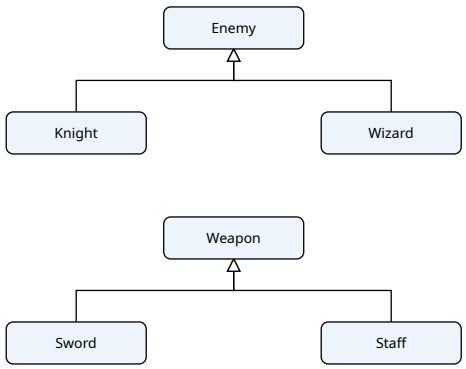


Figure 1: One way to organise the classes. Arrows with open triangles point from a class to the class it inherits from.

3.1. Output

Your wording and spacing do not need to match the example run below, but the information the criteria ask for must be visible on screen. The player types one answer per line.

4. Example Run

```
$ javac *.java && java Main
Turn Based RPG
--------------
[1] Play
[2] Quit
1
Steve's weapons:
  sword: 1 Damage
  staff: 1 Damage

Steve sees the knight!
  knight: 4 HP, 1 Damage

--- Steve's turn (3 HP) ---
What do you do?
  [0] Attack knight: 4 HP, 1 Damage
> 0
With which weapon?
  [0] sword: 1 Damage
  [1] staff: 1 Damage
> 1
Steve attacks knight with the staff
  knight: 3 HP, 1 Damage

--- knight's turn ---
knight asks: What is 12 - 45?
> 57
Wrong! The answer was -33.
knight attacks Steve for 1 damage!
Steve has 2 HP left
```

The exact numbers in the math questions differ each run because they are generated randomly.

5. Criteria

Your program is graded by playing it. The duplication criterion is graded by reading your code.

Criteria	Points
The program compiles with the unmodified `Main` and runs a whole game without crashing.	10
Minimal duplication. Behaviour that `Knight` and `Wizard` share is written once and not copied between them (8 points). The same goes for `Sword` and `Staff` (7 points). Similar classes should differ only in the few things that really differ.	15
At the start, both weapons are listed with their damage. Each enemy sighting shows the enemy's name, HP, and damage.	10
The player's turn shows numbered menus for the enemy and the weapon, and defeated enemies disappear from the enemy menu.	10
Weapon rules hold: the staff hurts only knights and the sword hurts only wizards, the attack is announced, and using the wrong weapon prints a message and does no damage.	10
When an enemy is defeated a message is shown, and it takes no more turns.	10
Knights ask subtraction questions (0 to 99) and wizards ask multiplication questions (0 to 9). Each turn has a new random question.	10
A correct answer means no damage. A wrong answer costs the player 1 HP and shows the HP remaining.	10
When the player reaches 0 HP, HP never shows below 0, no more questions are asked, and „You lose…“ is printed.	10
Wave 1 is one knight, wave 2 is a knight and two wizards, and defeating both waves prints „You win!!!“.	5
Input is handled cleanly with a single shared `Scanner`.	BONUS 10

Criteria	Points
The program has clear documentation including the author's name, student number, and date.	BONUS 10

6. Tips

- A knight and a wizard are almost the same: they have a name, HP, and damage, they can be attacked, and they take turns asking questions. Ask yourself what is the same, what differs, and where the shared part should live. The same goes for the sword and the staff.
- A subclass constructor can pass values up to its superclass with `super(...)`. It must be the first statement.
- `Random.nextInt(n)` gives a value from `0` to `n - 1`.

```java
Random rand = new Random();
int first = rand.nextInt(100);   // 0 to 99
int second = rand.nextInt(10);   // 0 to 9
```

- If you compare names as strings (for example to decide which enemy a weapon can hurt), spelling and capitalization matter. `"wizard"` does not match `"Wizard"`.
- After an attack defeats an enemy, its HP may drop to 0 or below. Remove it from `visibleEnemies` so it no longer acts.
- Enemies keep taking turns until the end of the wave, even after the player is dead. Check the player's HP before an enemy acts.
- Keep using the provided `sc` for every `Scanner` read inside your classes.
- Stuck? Most criteria have an entry in the hint catalog. Hints cost points.

7. Why is the Scanner passed in?

You might be tempted to write `Scanner sc = new Scanner(System.in);` inside every method. A `Scanner` wraps an input stream and buffers what it reads. When you create a new `Scanner` around `System.in` several times, one of them can consume buffered input that the others never see, and the symptoms are confusing (input that is skipped or missing). Creating one `Scanner` in `Main` and passing it to `playerTurn` and `enemyTurn` avoids the problem entirely and keeps all input flowing through a single place.